

Code You Didn't Write

Understanding Large and Unfamiliar Codebases

CHRIS CROFT-WHITE
& MIKE SHAH

Code You Didn't Write

Understanding Large and Unfamiliar Codebases

Chris Croft-White
Staff Solutions Architect
chris@undo.io



Web: mshah.io
YouTube: www.youtube.com/c/MikeShah
Social: mikesah.bsky.social
Courses: courses.mshah.io
Talks: <http://tinyurl.com/mike-talks>

60 minutes | Intermediate Audience
13:30 - 14:30 Friday, Sept. 18, 2026

Abstract (which you already read :))

Talk Abstract: First day on a new project. You set up your workstation and then download a repository of over 1,000,000 lines of code. Even more intimidating, the code has been around for 20 years. Parts are in legacy C++, parts use the latest C++26 features, and increasingly, AI coding assistants are weaving new code through all of it, meaning some parts of the codebase may have never been fully understood by a developer. You feel overwhelmed. **Don't Panic!** In this talk, we share techniques and tools that real engineers use to find their bearings in code they didn't write, from the humble grep through debuggers and profilers to time-travel debugging and AI-assisted exploration. The audience will leave this talk with a mix of simple and advanced tricks for navigating large and complicated codebases.

<https://cppcon2026.sched.com/event/2RT88/code-you-didnt-write-understanding-large-and-unfamiliar-code-bases>

**DON'T
PANIC**

Your Tour Guide(s) for Today

Chris Croft-White

- **Current Role:** Staff Solutions Architect at [Undo.io](https://undo.io) (6 years)
 - Supporting new customers and creating content to help people debug software
 - Background: ~20 years a pre-sales engineer
- **Available for:**
 - **Agentic Time Travel Debugging**, happy to explain the technology, its uses, and get you started
 - **Sleep**, it's dreadfully lacking and top of the list if I ever get any of this spare time I keep hearing about
- **Fun:**
 - Computer games, travelling, food & drink, learning cool new technologies, meeting new people.



Your Tour Guide(s) for Today

Mike Shah

- **Cppcon Role:** co-chair of Back to Basics and Software Development tracks for last 5 years
- **Current Role:** Teaching Faculty at **Yale University**
(Previously Teaching Faculty at Northeastern University)
 - **Teach/Research:** computer systems, graphics, geometry, game engine development, and software engineering.
- **Available for:**
 - **Contract work** in Gaming/Graphics Domains
 - e.g. tool building, plugins, code review
 - **Technical training** (virtual or onsite) in Modern C++, D, and topics in Performance or Graphics APIs
- **Fun:**
 - Guitar, running/weights, traveling, video games, and cooking are fun to talk to me about!



Web

www.mshah.io



YouTube

<https://www.youtube.com/c/MikeShah>

Non-Academic Courses

courses.mshah.io

Conference Talks

<http://tinyurl.com/mike-talks>

Question to the Audience:

What's the biggest codebase you have ever worked on?
(i.e. the approximate lines of code)

What's the biggest codebase you have ever worked on? (2/3)

Follow-up question(s):

1. Did that codebase ever get smaller?
2. Did that codebase become less complex over time?
3. How well was that codebase documented?
 - a. i.e. Could I go find tutorials, internal wiki pages, or internal/external videos explaining the code or architecture?
4. Did anyone say “Scrap the project -- let's rebuild it from scratch”?

What's the biggest codebase you have ever worked on? (3/3)

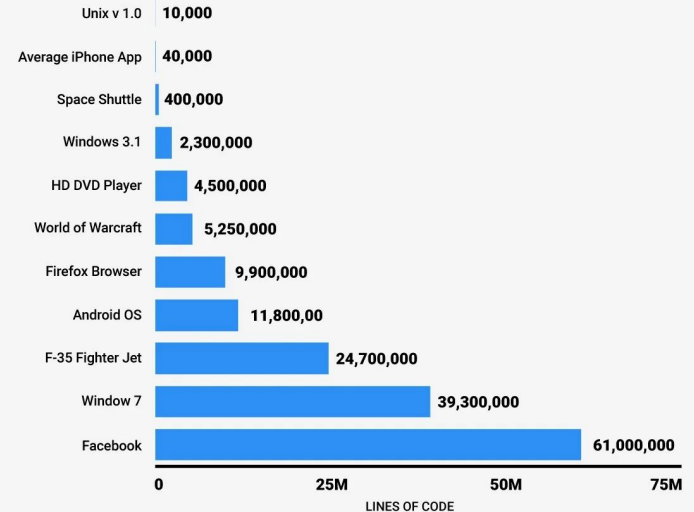
Follow-up question(s):

1. Did that codebase ever get smaller?
2. Did that codebase become less complex over time?
3. How well was that codebase documented?
 - a. i.e. Could I go find tutorials, internal wiki pages, or internal/external videos explaining the code or architecture?
4. Did anyone say “Scrap the project -- let's rebuild it from scratch”?
 - a. ^Sometimes we feel this way -- but it can be hard to convince a business to throw away thousands/millions of lines of code :)

How Big are *Big Codebases*?

- Today we are talking about **large** and **unfamiliar** codebases -- which contains a majority of **code you did not write**
 - The scale of code we are interested in is anything from tens of thousands of lines, to billions of lines of code (e.g. monorepo)
 - i.e. Big enough you cannot keep the whole design in your head

HOW MANY LINES OF CODE MAKE UP THESE POPULAR TECHNOLOGIES



SOURCE: NASA, Quora, Ohloh, Wired

BUSINESS INSIDER

This figure gives a very **rough** approximation of various codebase sizes.

Figure source: https://external-preview.redd.it/8P66ds7y-3j7n-CEfMh_mE6n7G8vrb08wV05Q62v_M-8o7auewepb04ech817211341210e5d5a26f238726d4e864783d4
Wired Article for more on code sizes: <https://www.wired.com/2015/09/opoole-2-billion-lines-code-and-one-place/>

Getting started

- This talk is about understanding code you didn't write
 - Code you wrote last year may as well have been written by someone else (it was)
- We won't talk about how to write code
- We will cover how the team can help, whether or not you are joining a new team while picking up someone else's code
- Yes, AI will be part of this talk, but as a tool and certainly not the solution to everything

The big questions

Who knows what?

Why is it like this?

What does it actually do?

Whether you are talking about a code base or a team

Who knows what?

Why is it like this?

What does it actually do?

Human factors can additionally contribute to making your life harder

Who knows what? - The team and the first day

Newcomers are often wasted assets

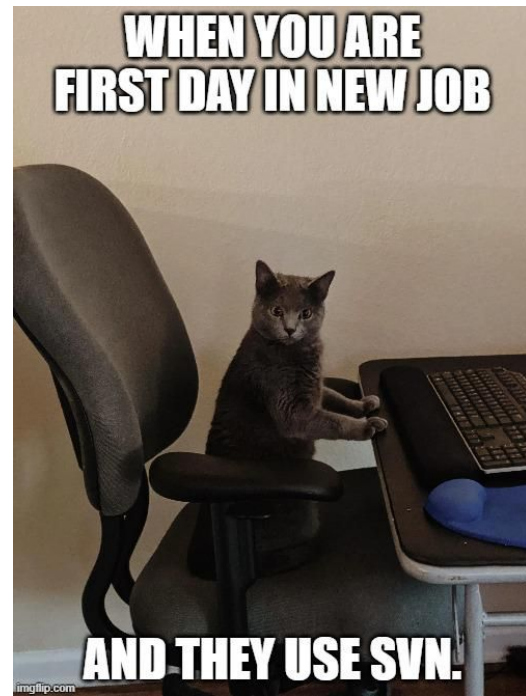
- We ask new eyes to evaluate OOBE (Out-of-Box Experience), but not the team onboarding experience
- Product feedback has clear owners, team feedback needs the same
- Newcomers should be empowered to give feedback, not made responsible for fixes. Senior team members should visibly action the feedback. Avoid delegating back to the newcomer
- Try to capture the experience, not just the result, and preserve this context

Question to the Audience:
Why do folks think I'm talking about this topic?

Why do folks think I'm talking about this topic? (2/3)

- **Answer:** It was an honest difficulty of mine when I started out as a software engineer!

- (more on next slide)



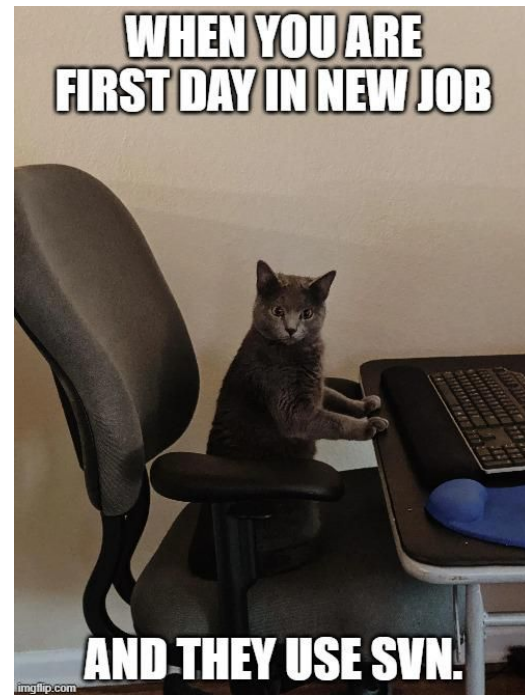
Ah, I have fond memories of TortoiseSVN -- it's still active as far as I know!

<https://preview.redd.it/whatdididont-vd-3v70uo8hdvc1-lineo2width=320&crop=smart&auto=webp&s=5d56747737e31e0dfba5ca86ce9fde9a9f173a>

Why do folks think I'm talking about this topic? (3/3)

- **Answer:** It was an honest difficulty of mine when I started out as a software engineer!

- I remember the difficulty and overwhelming feeling as an intern learning new tools & code!
 - (On occasion I consult on short contracts -- so this feeling still occurs!)
 - I suspect there are others in the audience who remember this daunting feeling too
- It probably feels a bit embarrassing to admit that feeling on the first day of work!
 - We should not be afraid to ask questions however!



Ah, I have fond memories of TortoiseSVN -- it's still active as far as I know!

<https://preview.redd.it/whatdididont-vd-3v79uo8hdvc1-1neo2widhe320&crocs=marl&auto=webp&s=456747737e31e07fba5ca6c6edfde9f173a>

Where do we learn to read code?

- The other reason is that we are not always taught many tools or *how to read large codebases* in school or our workplace
 - **Let me be clear -- I'm not necessarily blaming schools or your workplace** -- there is a lot of computer science to cover and remember.
 - Some large projects in school are maybe 2-5k lines of code -- but again not quite the scale we are talking about for millions of lines of code
 - **Note:** I (and many other professors) do often expose students to large codebases at various points -- probably some professors out there doing an excellent job as well!

./missing-semester | lectures | about

The Missing Semester of Your CS Education

Classes teach you all about advanced topics within CS, from operating system to machine learning, but there's one critical subject that's rarely covered, and instead left to students to figure out on their own: proficiency with their tools. We'll teach you how to master the command-line, use a powerful text editor, use fancy features of version control systems, and much more!

Students spend hundreds of hours using these tools over the course of their education (and thousands over their career), so it makes sense to make the experience as fluid and frictionless as possible. Mastering these tools not only enables you to spend less time on figuring out how to bend your tools to your will, but it also lets you solve problems that would previously seem impossible or complex.

Read about the [motivation behind this class](#).

Note: Yale, MIT, Tufts, UMich, my own courses and many other universities do make efforts to teach various useful tools and often in dedicated courses. <https://missing.csail.mit.edu/>

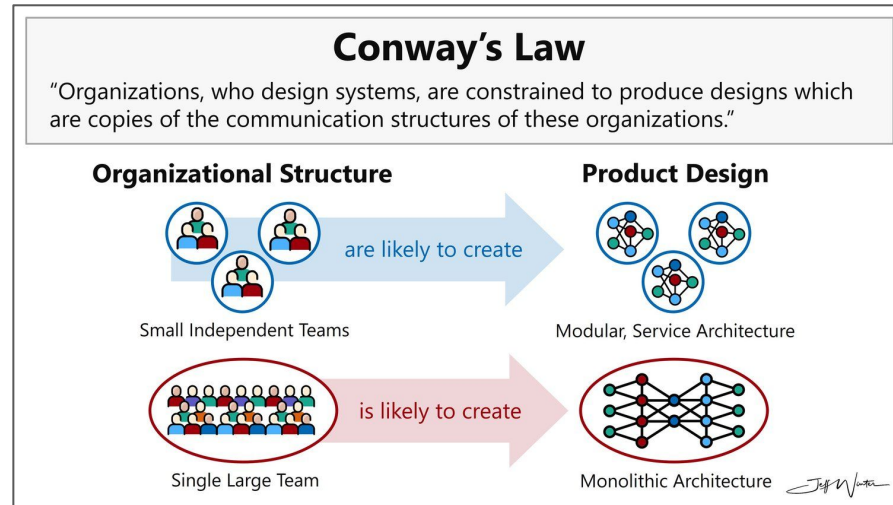
As you read code (written by a team)--Ask for Help

- "If you don't know, just ask"
 - **If you're a junior engineer**, then bring what you have tried/learned for the problem you have solved
 - **If you're a senior engineer**, make a note of where recurring questions may be coming from in the project.
 - Generally, don't put juniors in uncomfortable positions either -- sit down with them when they first join your team to help them get acquainted!
 - Screen record your session (if appropriate), or let junior engineers take a picture of the whiteboard
 - Seniors documenting 'who knows what tribal knowledge' can also be very helpful.
- Don't be arrogant, and always be nice to your teammates.
 - Empathy matters **a lot** when working on teams, and it helps when you want to ask each other questions!

How to figure out who to ask
and why it's not as easy as those who already know might think

Team and code structure

- **Human factors** can additionally contribute to making your life harder
 - Team size / structure
 - Accessibility to code that you may not own.
 - (Is all the code in the same repo, or do you need permission to view other libraries?)
 - No version history or documentation of the 'why' something evolved
 - Undocumented code that only a 'senior engineer' understands, and everyone is afraid to touch.



Ever heard of Conway's law? Organizational structure can potentially impact the very structure of code itself!

<https://www.jeffwinterinsights.com/insights/conways-law-enterprise-architecture>

Context is important

(Aside) Human Factors: Our Memory

- For a point of reference:
 - ***“A million lines of code, if printed, would be about 18,000 pages of text. That’s 14x the length of War and Peace.”***
- That’s a lot of ‘text’, architecture, subsystems, etc. to keep in our head!
 - So -- how easy is it for you to recall a specific detail on page 47, paragraph 6?
 - Even the author of the book probably does not remember all the text they have written.
 - **So code you wrote years ago -- it may as well be someone else's or some other organization's code.**



Figure source:
https://stackoverflow.com/image/fetch?_auto_g_auto&good=1&progressive=steep&https%3A%2F%2Fsubstack-com%2Fmedia-s3.amazonaws.com%2Fpublic%2Fimages%2F6b32b94-f99e-498f-840b-4d789c6503c-1536x2048.jpeg

Demonstration with the Audience:

We will need a volunteer.

Action Item for Senior Developers

- For your organization -- **create a ‘first day/week in project X’ document.**
- Don’t ‘outsource’ this to an agent
 - Recall: This is the **‘human factors’** section of the talk
 - Actually sit down and think when you hire someone -- what are they working on, what tools/pointers would have been helpful for you, what’s the organizational structure
 - Things that you struggled/suffered with -- do your best to make sure someone else does not have to struggle/suffer with them.
 - Avoid "I had to struggle, now it's your turn", it is not how they learn, be better
- **This task is not ‘soft skills’ or ‘optional work’ -- this is engineering**
 - There’s an input (a junior engineer or new team member with this document in hand) and an output (a more productive happy engineer)

We should also erase the term ‘soft skills’ from our lexicon

How AI can help

- Humans own the documentation, but AI can help collect the data.
 - The context is out there, just people don't have time to collate it.
- Have AI read your Slack (or similar) and email history:
 - Check your policies first to be sure this is allowed
 - Look for questions asked about the code or team organisation / processes
 - Generally, but especially by new colleagues in their first few months
 - Look for who answers them and extract domain expertise from this
 - Ask git the same question and compare. Who talks about an area and who commits to it are different people
 - Have the AI build the mapping, and identify gaps
 - "Trust but verify" - find ways to ensure the outcome is valid
- I tried this last weekend and got a good initial knowledge map of Undo in <15 minutes (and <\$6)
 - Undo's policies allowed this and the data shared with AI is contractually protected

AI can be used to capture what's human

Who knows what?

Why is it like this?

What does it actually do?

What was written down, and what wasn't

Two directions of lost context

- The people who wrote this knew things the code and docs do not record
- You cannot leave all the context for the next person, and it's hard to predict what is and isn't going to be needed
- Code you wrote last year was effectively written by somebody else
- Some of this code had no author to ask in the first place

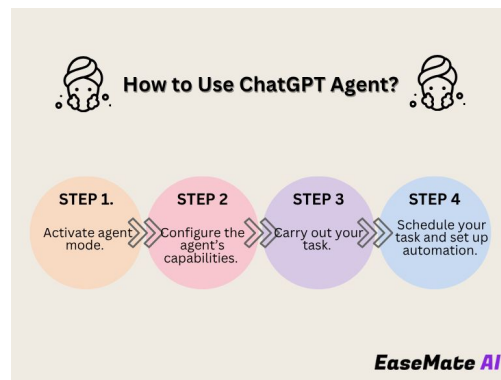
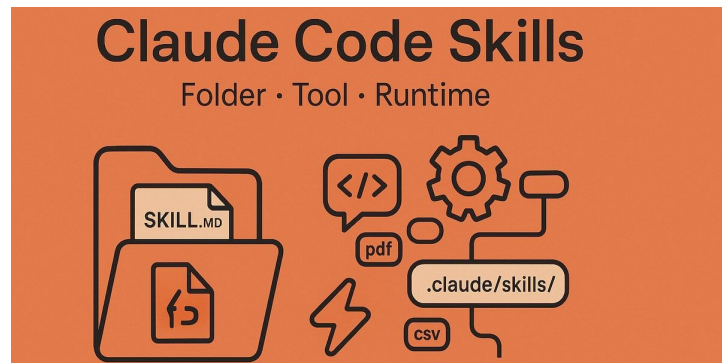
Neither direction is anyone's fault, it's just reality



What makes a codebase hard to understand? (1/2)

- **Non-Human factors**

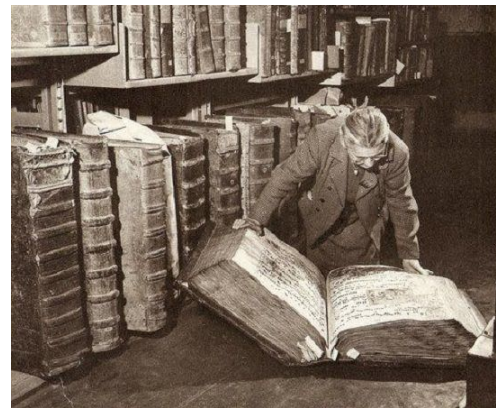
- Who exactly is writing the code these days?
- Depending on your workplace policies, AI Agents and AI-assisted tools may be writing code.
 - Sometimes the AI tools are different, and they are rapidly changing.



What makes a codebase hard to understand? (2/2)

- The **pure scale** of software and its relation to complexity
 - i.e. The number of lines of source code
- Different variations of code
 - e.g. Legacy C++ mixed with C++26
 - Different 'design patterns' or architecture design within the source code

```
1 // ...
2
3 // ...
4
5 // ...
6
7 // ...
8
9 // ...
10
11 // ...
12
13 // ...
14
15 // ...
16
17 // ...
18
19 // ...
20
21 // ...
22
23 // ...
24
25 // ...
26
27 // ...
28
29 // ...
30
31 // ...
32
33 // ...
34
35 // ...
36
37 // ...
38
39 // ...
40
41 // ...
42
43 // ...
44
45 // ...
46
47 // ...
48
49 // ...
50
51 // ...
52
53 // ...
54
55 // ...
56
57 // ...
58
59 // ...
60
61 // ...
62
63 // ...
64
65 // ...
66
67 // ...
68
69 // ...
70
71 // ...
72
73 // ...
74
75 // ...
76
77 // ...
78
79 // ...
80
81 // ...
82
83 // ...
84
85 // ...
86
87 // ...
88
89 // ...
90
91 // ...
92
93 // ...
94
95 // ...
96
97 // ...
98
99 // ...
100
```



'source code' we visualize as text -- printing out the code will look something like the bottom-right image.

Technical reality not technical debt

- Most code was the best available answer to the constraints at the time
- The past developers did not have the context you have today, on how the code would be used or what requirements would change
- "Why was it done this way?" is a question asked with information they did not have, about working under constraints you don't know
- Debt is often seen as a negative thing, however, usually there were good reasons for the decisions that are causing you problems today

Given:

An unknown (open-source) codebase



Real World Example: Transport Tycoon

- We will look at the video game **Transport Tycoon**, or rather its open source successor, **OpenTTD**
- Some of you may be familiar with it, and it is perhaps better if you are not!
 - Although, I will say it's a wonderfully charming game...
- A good project to practise on is open source, long lived, written by many hands at different levels, and ideally one you do not already know



You can pick up Open Transport Tycoon here:
<https://www.openttd.org/>

(Aside) Project Scale: 618,852 lines of code

- Let's agree this is a large size codebase to investigate!
 - 'cloc' (count lines of code)** tool counts lines of code and provides some insights such as if multiple languages are used
- The stats for /src in the Open Transport Tycoon game are:
 - 191,086 C++
 - 120,496 C/C++ headers
 - etc.

```
chris@hyperion:~/Undo/Demos/OpenTTD/src$ cloc .
1523 text files.
1509 unique files.
14 files ignored.

github.com/AlDanial/cloc v 1.98 T=1.60 s (940.6 files/s, 521893.1 lines/s)
-----
Language                      files      blank      comment      code
-----
Text                           67         54915          0       298417
C++                           558        41844       50622       191086
C/C++ Header                   805        24091       44768       120496
INI                             20          576         151         4443
CMake                          48           85           9         1960
Objective-C++                   4          466        627         1643
Markdown                       4          148           0          541
awk                             1           43          34          229
reStructuredText                1            5           0           22
Bourne Shell                    1            6           6           15
-----
SUM:                          1509       122179      96217      618852
-----
```



x9

Entry Points (1/3)

- One of the first things I like to do is to anchor around the 'main()' function
- From the main() function, you will often get hints as to the overall software architecture
 - (i.e. are there plugins loaded, is there an event loop, etc.)
 - Some things you can actively log are:
 - What happens before main (any initialization code)
 - What happens after (cleanup code)

```
21 int CDECL main(int argc, char *argv[])
22 {
23     /* Make sure our arguments contain only valid UTF-8 characters. */
24     for (int i = 0; i < argc; i++) StrMakeValidInPlace(argv[i]);
25
26     CrashLog::InitialiseCrashLog();
27
28     SetRandomSeed(time(nullptr));
29
30     signal(SIGPIPE, SIG_IGN);
31
32     return openttd_main(argc, argv);
33 }
```

In my opinion, a good software engineering practice is to keep your 'main' relatively clean. Observe here, the main entry point returns to the 'real main (openttd_main)'

In this particular software, you'll observe there are also different 'main' functions per operating system.

Entry Points (2/3)

- Here's an example of finding 'main()' with **grep**
- [grep](#) is a tool for searching for patterns within files
 - e.g.
 - `grep -i -r "main"`
 - This does a case insensitive(**i**) search, recursively through the directory (**r**), and provides line numbers (**n**) with the filenames
 - This gives us around **37 places to look**

```
mike@mike-MS-7B17:openttd-14.1$ grep -i -r "main()" .
grep: ./build/openttd: binary file matches
grep: ./build/openttd test: binary file matches
./build/CMakeFiles/_CMakeLT0Test-CXX/src/main.cpp:3:int main()
./build/CMakeFiles/3.28.3/CompilerIdC/CMakeCCompilerId.c:848:void main() {}
grep: ./build/CMakeFiles/openttd_lib.dir/src/video/opengl.cpp.o: binary file matches
./os/emscripten/cmake/FindLibLZMA.cmake:8: int main() { return 0; }"
./github/unused-strings.py:184:def main():
./github/unused-strings.py:220: main()
./github/script-missing-mode-enforcement.py:53:def main():
./github/script-missing-mode-enforcement.py:71: main()
./src/table/opengl_shader.h:17: "void main() {",
./src/table/opengl_shader.h:32: "void main() {",
./src/table/opengl_shader.h:45: "void main() {",
./src/table/opengl_shader.h:56: "void main() {",
./src/table/opengl_shader.h:67: "void main() {",
./src/table/opengl_shader.h:80: "void main() {",
./src/table/opengl_shader.h:115: "void main() {",
./src/table/opengl_shader.h:138: "void main() {",
./src/table/opengl_shader.h:164: "void main() {",
./src/table/opengl_shader.h:191: "void main() {",
./src/os/macosx/macos.mm:211: * These are called from main() to prevent a _NSAutoreleaseNoPool error when
./src/openttd.cpp:123: * openttd main() is never executed. */
```

Entry Points (3/3)

- **grep** is quite powerful, so you will want to spend some time playing with it
- Here's a more specific search that gives us around 7 **places to look**
 - `grep -irn -I --include=*.cpp "main()" .`
 - `-I` tells us to ignore binary files
 - `--include=*.cpp` tells us to look at .cpp files

```
mike@mike-MS-7B17:openttd-14.1$ grep -irn -I --include=*.cpp "main()" .
./build/CMakeFiles/_CMakeLT0Test-CXX/src/main.cpp:3:int main()
./src/openttd.cpp:123: * openttd_main() is never executed. */
./src/landscape.cpp:1322: finder.Main();
./src/video/sdl2_v.cpp:623: * openttd_main() is never executed. */
./src/3rdparty/md5/md5.cpp:46: 2002-03-11 lpd Corrected argument list for main(), and added int return
./src/pathfinder/npf/aystar.cpp:245:int AyStar::Main()
./src/pathfinder/npf/npf.cpp:1044: [[maybe_unused]] int r = _npf_aystar.Main();
```

Find a first good task (1/2)

- With a large codebase, we might get some direction by literally just solving a problem.
 - If you're lucky, your code base will be labeled with 'good first tasks'
 - Good first tasks are 'byte sized' -- i.e. small, and picked out by engineers on your team
- For our project today -- observe there is a **Development** tab on the website.
 - <https://www.openttd.org/development>

The screenshot shows the OpenTTD website's 'Development' page. At the top, there are download links for stable (15.3), testing (16.0-beta2), and nightly (20260911) versions. The navigation bar includes links for Home, About, Wiki, Screenshots, Servers, Development (which is highlighted), Forum, Community, Contact, and Donate. The main content area has three sections: 'Translating', 'What to do When You Find a Bug', and 'Obtaining The Source Code'. The 'Translating' section asks for help with translation and provides links to the GitHub translation team and a web translator. The 'What to do When You Find a Bug' section lists steps for reporting bugs, such as running the latest version, making the bug reproducible, and providing savegames or crash logs. The 'Obtaining The Source Code' section explains that the project uses Git and provides the GitHub repository URL.

Download stable (15.3)
Download testing (16.0-beta2)
Download nightly (20260911)

Home About Wiki Screenshots Servers **Development** Forum Community Contact Donate

Translating

If you would like to help translating the game, please

- [join our translation team on GitHub](#) (you will need a GitHub account).
- After joining you get access to [our Web Translator](#).

What to do When You Find a Bug

When you think you found a bug in OpenTTD you should:

- Make sure you are running a recent version, i.e. run the latest stable, testing or nightly based on where you found the bug.
- Make sure you are not running a non-official binary, like a patch pack. If you are playing with a patch pack you should report any bugs to the forum thread related to that patch pack.
- Make it reproducible for the developers. In other words, create a savegame in which you can reproduce the issue once loaded. For Windows users it is very useful to give us the crash.dmp and crash.log which are created on crashes.
- Check whether the bug is already reported on our [bug tracker](#). This includes searching for recently closed bug reports as the bug might already be fixed.
- When you are certain it is an unknown bug you should make a bug report at our [bug tracker](#). The bug report must contain a savegame, preferably crash.sav and the last (automatically) saved savegame. It must also contain crash.log and crash.png (if they were created). For crashes on Windows must attach the crash.dmp files too. For bugs in nightlies you must also specify the revision of the nightly.

Obtaining The Source Code

For OpenTTD we use the [git](#) version control system. OpenTTD's source code repository using git is accessible by anyone on the following URL:

- <https://github.com/OpenTTD/OpenTTD>

You can also [browse](#) the repository online. The [documentation](#) of the source is created by DoxyGen.

To download the source code you need a git client. See [Getting Started](#) for more information about git and how to use it.

Find a first good task (2/2)

- Searching for 'bugs' or 'issues' is a great place to start, often they will be labeled
- Two labels imply a small amount of work, probably a good place to start

OpenTTD / OpenTTD

Q Type [Z] to search

<> Code Issues 188 Pull requests 135 Discussions Actions Security Insights

Want to contribute to OpenTTD/OpenTTD?

If you have a bug or an idea, read the [contributing guidelines](#) before opening an issue.
If you're ready to tackle some open issues, [we've collected some good first issues for you](#).

is:issue state:open

Labels Milestones New issue

Open 188 Closed 8,391

Author Labels Projects Milestones Assignees Types Newest

- [Crash]: Video driver install triggers fatal blitter assertion "_screen.dst_ptr != nullptr"
#14350 · GreenReaper opened 6 hours ago
- [Bug]: German industry names should be [industry] [town name]
#14342 · RiedlerO opened 3 days ago
- [Bug]: Companies in multiplayer can be in the red for years before going bankrupt **needs triage**
#14328 · bjornwarmedal opened last week

- size: large
- size: small
- size: trivial
- stale
- waiting on author
- waiting on runner update
- wip
- won't fix
- won't implement
- work: minor details

<https://github.com/OpenTTD/OpenTTD/issues>

If you are not assigned a good ‘first task’:

- Don't forget about our tool ‘grep’
 - Try searching for ‘TODO’, ‘FIXME’ or ‘Later’ as ‘keywords’
 - If you're lucky, you may find notes something like:
 - “This is ‘hard-coded’ and should be fixed later”
 - “FIXME, this is a ‘magic constant’ that should be loaded from a config file...”
 - etc.
- `grep -irn -I --include=*.cpp "TODO" ./src/`

```
mike@mike-MS-7B17:openttd-14.1$ grep -irn -I --include=*.cpp "TODO" ./src/
./src/music/midifile.cpp:1069: std::string tmpdirname = FioGetDirectory(Searchpath::SP_AUTODOWNLOAD_DIR);
./src/strgen/strgen_base.cpp:31: int _errors, _warnings, _show_todo;
./src/strgen/strgen_base.cpp:363: if ((_show_todo & 2) != 0) StrgenWarning("{}");
./src/strgen/strgen_base.cpp:753: /* Abusing _show_todo to replace "warning" with "info" for translation
./src/strgen/strgen_base.cpp:754: _show_todo &= 3;
./src/strgen/strgen_base.cpp:755: if (!this->translation) _show_todo |= 4;
./src/strgen/strgen_base.cpp:929: if (_show_todo > 0 && ls->translated.empty()) {
./src/strgen/strgen_base.cpp:930: if ((_show_todo & 2) != 0) {
./src/strgen/strgen_base.cpp:933: if ((_show_todo & 1) != 0) {
./src/strgen/strgen_base.cpp:934: const char *s = "<TODO> ";
./src/strgen/strgen.cpp:38: if (_show_todo > 0) {
./src/strgen/strgen.cpp:318: GETOPT_NOVAL( 't', "--todo"),
./src/strgen/strgen.cpp:371: _show_todo |= 1;
./src/strgen/strgen.cpp:375: _show_todo |= 2;
./src/strgen/strgen.cpp:381: " -t | --todo replace any untranslated
./src/strgen/strgen.cpp:462: if ((_show_todo & 2) != 0) {
./src/newgrf_engine.cpp:287: /* @todo Need to find which terminal (or hangar) we've chosen
./src/newgrf_engine.cpp:297: /* @todo Need to check terminal we're landing to. Is it
./src/townname.cpp:192: * TODO: Perhaps we should use it for all the name generators? --pasky */
./src/tunnelbridge_cmd.cpp:11: * @todo separate this file into two
./src/os/windows/win32.cpp:374: _searchpaths[SP_AUTODOWNLOAD_PERSONAL_DIR] = tmp;
./src/fileio.cpp:98: case SP_AUTODOWNLOAD_DIR: // Otherwise we cannot download
./src/fileio.cpp:862: _searchpaths[SP_AUTODOWNLOAD_PERSONAL_DIR_XDG] = tmp;
./src/fileio.cpp:872: _searchpaths[SP_AUTODOWNLOAD_PERSONAL_DIR_XDG] = tmp;
./src/fileio.cpp:875: _searchpaths[SP_AUTODOWNLOAD_PERSONAL_DIR_XDG].clear();
./src/fileio.cpp:891: _searchpaths[SP_AUTODOWNLOAD_PERSONAL_DIR] = tmp;
./src/fileio.cpp:894: _searchpaths[SP_AUTODOWNLOAD_PERSONAL_DIR].clear();
./src/fileio.cpp:1078: /* If we have network we make a directory for the autodownloading of content */
./src/fileio.cpp:1079: _searchpaths[SP_AUTODOWNLOAD_DIR] = personal_dir + "content download" PATHSEP;
./src/fileio.cpp:1080: Debug(misc, 3, "{} added as search path", _searchpaths[SP_AUTODOWNLOAD_DIR]);
./src/fileio.cpp:1081: FioCreateDirectory(_searchpaths[SP_AUTODOWNLOAD_DIR]);
./src/fileio.cpp:1087: FioCreateDirectory(FioGetDirectory(SP_AUTODOWNLOAD_DIR, dirs[i]));
./src/economy.cpp:1815: /* TODO: Regarding this, when we do gradual loading, we
./src/economy.cpp:2038: * @todo currently this only works for AI companies
./src/order_gui.cpp:1521: * TODO: give a warning message */
./src/order_gui.cpp:1807: * TODO Rewrite the order GUI to not use different WindowDescs.
```

No docs? Generate them

- Most projects have no architecture documentation, and the docs that exist describe what the code did some time ago
- You can generate a readable map of the code as it is today
- `doxygen -g` then `doxygen Doxyfile`, and you have a browsable site in a couple of minutes
- Doxypress and other generators do the same job if you prefer them

Code Documentation.
Automated.

✎ Software documentation consulting

Free, open source, 🔍 cross-platform.

<https://www.doxygen.nl/>

For more configuration options check:

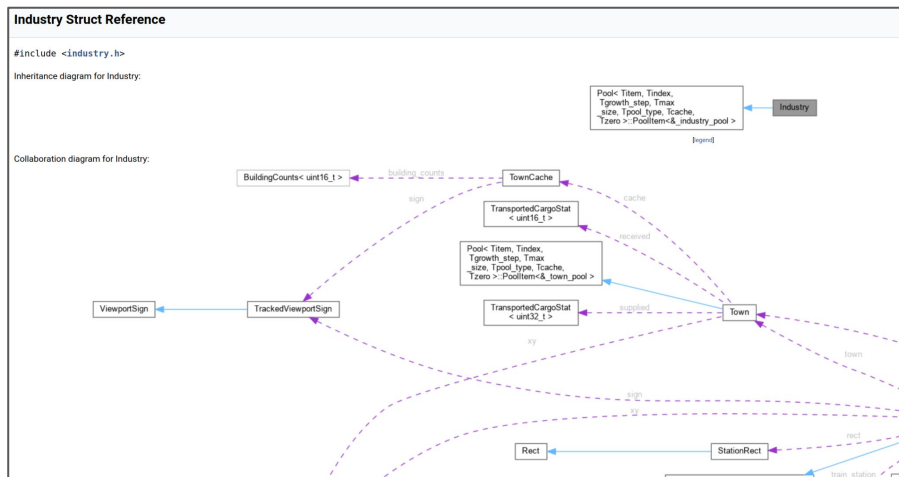
- <https://youtu.be/tLPHQMosF9M>

What the generated docs tell you

- Naming conventions, so you can guess at names before you grep for them
- How errors travel, return codes or exceptions, which dates the code
- Inheritance and collaboration diagrams, which show which types are central

What it cannot give you is why any of it is that way

Classes
struct AcceptedCargo
struct ProducedCargo
struct ProducedHistory
Public Types
using ProducedCargoArray = std::array< ProducedCargo, INDUSTRY_NUM_OUTPUTS >
using AcceptedCargoArray = std::array< AcceptedCargo, INDUSTRY_NUM_INPUTS >
Public Types inherited from Pool< Titem, Tindex, Tgrowth_step, Tmax_size, Tpool_type, Tcache, Tzero >::PoolItem<&_industry_pool >
Public Member Functions
Industry (TileIndex tile=INVALID_TILE)
void RecomputeProductionMultipliers ()
bool TileBelongsToIndustry (TileIndex tile) const
ProducedCargoArray::iterator GetCargoProduced (CargoID cargo)
ProducedCargoArray::const_iterator GetCargoProduced (CargoID cargo) const
AcceptedCargoArray::iterator GetCargoAccepted (CargoID cargo)
bool IsCargoAccepted () const
bool IsCargoProduced () const
bool IsCargoAccepted (CargoID cargo) const
bool IsCargoProduced (CargoID cargo) const
const std::string & GetCachedName () const
Public Member Functions inherited from Pool< Titem, Tindex, Tgrowth_step, Tmax_size, Tpool_type, Tcache, Tzero >::PoolItem<&_industry_pool >
Static Public Member Functions
static Industry * GetByTile (TileIndex tile)

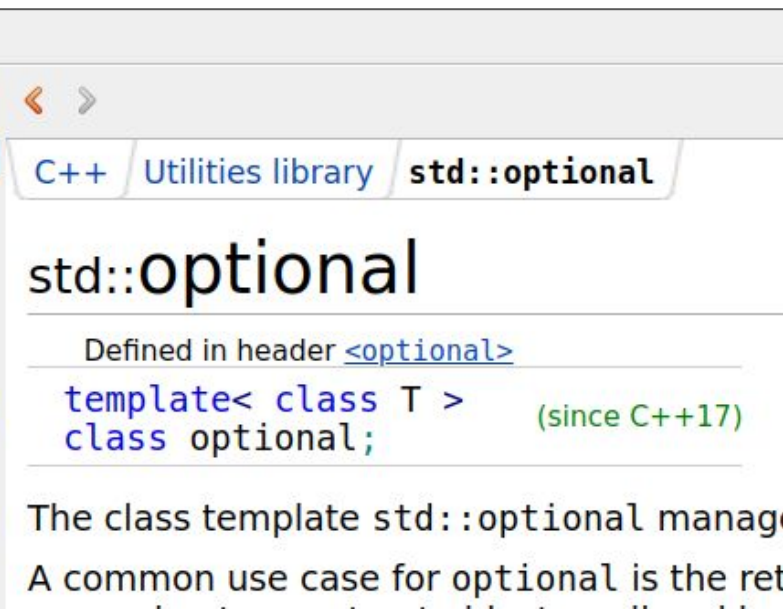
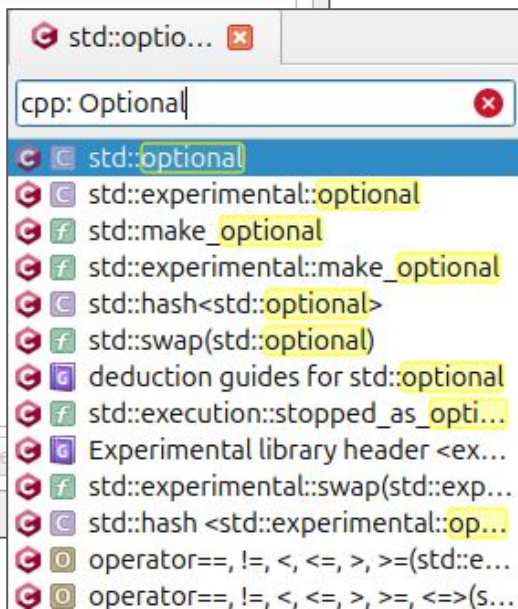
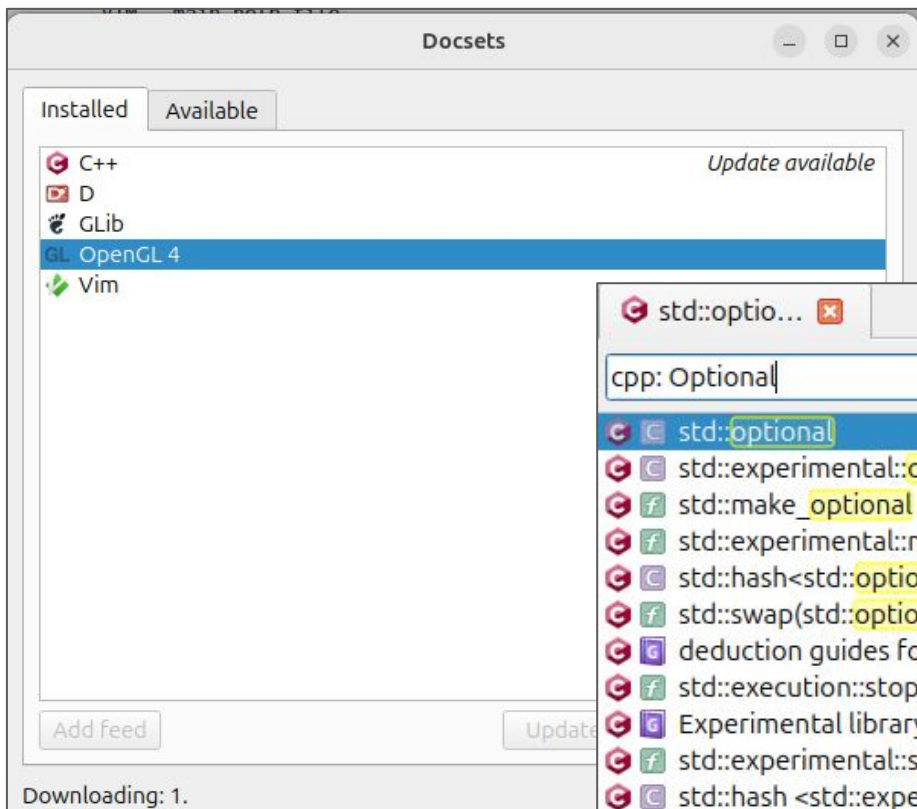


In a big project, there is code that you do not own

- ldd tells you what shared libraries are actually *linked*
 - (Dependency Walker does a similar task on Windows)
- If you are lucky the source is available, but some of it will be closed and you can only see the behaviour, not the code

```
mike@mike-MS-7B17:build$ ldd openttd
linux-vdso.so.1 (0x00007ffcc2dec000)
libpng16.so.16 => /lib/x86_64-linux-gnu/libpng16.so.16 (0x0000705b9a3b1000)
libz.so.1 => /lib/x86_64-linux-gnu/libz.so.1 (0x0000705b9a395000)
liblzma.so.5 => /lib/x86_64-linux-gnu/liblzma.so.5 (0x0000705b9a363000)
libfluidsynth.so.3 => /lib/x86_64-linux-gnu/libfluidsynth.so.3 (0x0000705b9a27c000)
libSDL2-2.0.so.0 => /lib/x86_64-linux-gnu/libSDL2-2.0.so.0 (0x0000705b97824000)
libfreetype.so.6 => /lib/x86_64-linux-gnu/libfreetype.so.6 (0x0000705b97758000)
libfontconfig.so.1 => /lib/x86_64-linux-gnu/libfontconfig.so.1 (0x0000705b97707000)
libharfbuzz.so.0 => /lib/x86_64-linux-gnu/libharfbuzz.so.0 (0x0000705b975fa000)
libicu18n.so.74 => /lib/x86_64-linux-gnu/libicu18n.so.74 (0x0000705b97200000)
```

Zeal Docs (or Dash) - Fast offline documentation



Search for ‘Design Patterns’ you will recognise

- Yes, they’re not perfect, but traditional patterns like ‘observer’, ‘visitor’, etc. tend to show up.
 - These are found in the [‘Design Patterns’ or ‘Gang of Four’ book](#)
- Recognising one hints at what the author expected to change later
- Often writing a toy version of these patterns can help you understand the context in a larger system.
- In some cases, you can try to find an application that may be similar to what you’re developing, and learn about how that system is documented.
 - There may be some hints within software case studies, otherwise try looking at related software.
 - See: <https://aosabook.org/en/>

Give yourself room to read

- You will be reading a lot of text, so beg, borrow or steal a large screen, or two
- Dell's 27 inch 4K screens start around \$300
- One of the few real hardware advantages left, since build and test are usually remote
- Find a window manager or a multiplexer you are otherwise comfortable with
- Comparing two pieces of code side by side is how you notice what you missed

```
1 // A material that allows for multiple textures
2 module multitexturematerial;
3 import pipeline, materials, texture;
4 import glBindc, OpenGL;
5
6 /// Represents a material with multiple textures
7 class MultiTextureMaterial : Material {
8     Texture mTexture1;
9     Texture mTexture2;
10    Texture mTexture3;
11    Texture mTexture4;
12
13    /// Construct a new material for a pipeline, and load a texture for that
14    pipeline
15    this(string pipelineName,
16         string textureFilename1,
17         string textureFilename2,
18         string textureFilename3,
19         string textureFilename4) {
20        // delegate to the base constructor to do initialization
21        super(pipelineName);
22
23        mTexture1 = new Texture(textureFilename1);
24        mTexture2 = new Texture(textureFilename2);
25        mTexture3 = new Texture(textureFilename3);
26        mTexture4 = new Texture(textureFilename4);
27    }
28
29    /// TextureMaterial.Update()
30    override void Update() {
31        // Set our active shader graphics pipeline
32        PipelineUse(pipelineName);
33
34        // Set any uniforms for our mesh if they exist in the shader
35        if ("sample1" in mMiniformMap) {
36            gActiveTexture(GL_TEXTURE0, mTexture1.mTextureID);
37            glBindTexture(GL_TEXTURE_2D, mTexture1.mTextureID);
38            mMiniformMap["sample1"].Set(1);
39        }
40        if ("sample2" in mMiniformMap) {
41            gActiveTexture(GL_TEXTURE1, mTexture2.mTextureID);
42            glBindTexture(GL_TEXTURE_2D, mTexture2.mTextureID);
43            mMiniformMap["sample2"].Set(1);
44        }
45        if ("sample3" in mMiniformMap) {
46            gActiveTexture(GL_TEXTURE2, mTexture3.mTextureID);
47            glBindTexture(GL_TEXTURE_2D, mTexture3.mTextureID);
48            mMiniformMap["sample3"].Set(1);
49        }
50        if ("sample4" in mMiniformMap) {
51            gActiveTexture(GL_TEXTURE3, mTexture4.mTextureID);
52            glBindTexture(GL_TEXTURE_2D, mTexture4.mTextureID);
53            mMiniformMap["sample4"].Set(1);
54        }
55    }
56 }
57
```

```
1 // An example of a normal map material
2 module normalmapmaterial;
3 import pipeline, materials, texture;
4 import glBindc, OpenGL;
5
6 /// Represents a normal mapped material
7 // Some notes on "albedo" vs "diffuse" map for naming: https://www.a23d.co/v/
8 // Difference between albedo and diffuse maps
9 class NormalMapMaterial : Material {
10    Texture mTexture1;
11    Texture mTexture2;
12
13    /// Construct a new material for a pipeline, and load a texture for that
14    pipeline
15    this(string pipelineName, string textureFilename, string normalmapFilename) {
16        // delegate to the base constructor to do initialization
17        super(pipelineName);
18
19        mTexture1 = new Texture(textureFilename);
20        mTexture2 = new Texture(normalmapFilename);
21    }
22
23    /// TextureMaterial.Update()
24    override void Update() {
25        // Set our active shader graphics pipeline
26        PipelineUse(pipelineName);
27
28        // Set any uniforms for our mesh if they exist in the shader
29        if ("albedo" in mMiniformMap) {
30            gActiveTexture(GL_TEXTURE0, mTexture1.mTextureID);
31            glBindTexture(GL_TEXTURE_2D, mTexture1.mTextureID);
32            mMiniformMap["albedo"].Set(1);
33        }
34        if ("normalmap" in mMiniformMap) {
35            gActiveTexture(GL_TEXTURE1, mTexture2.mTextureID);
36            glBindTexture(GL_TEXTURE_2D, mTexture2.mTextureID);
37            mMiniformMap["normalmap"].Set(1);
38        }
39    }
40 }
41
42 =====normalmap and # 3 (shader debug info)=====
43 GL LINK STATUS = 1
44 GL ATTACHED SHADERS = 0
45 GL ACTIVE ATTRIBUTES = 2
46 0) type=vec3 name=position location:0
47 1) type=sampler2D name=albedo location:1
48 2) type=mat4 name=mat4 location:2
49 3) type=mat4 name=projection location:3
50 4) type=mat4 name=uvproj location:4
51
52 =====
53 Uniforms automatically parsed from ./pipelines/normalmap.vert.glsl
54 Uniforms automatically parsed from ./pipelines/normalmap/frag.glsl
55 Uniforms automatically parsed from ./pipelines/normalmap/frag.glsl
56 Uniforms automatically parsed from ./pipelines/normalmap/frag.glsl
57
58 Dimensions: 312, 512.
59 Camera Position: <0.0,0.0>
60 Pressed escape key and now exiting...
61 Camera Position: <0.0,0.0>
62 Shutting down SDL

```

Tools like tmux and VIM can split and manage your windows.

The **why** was never written down

- We industrialised the documentation of interfaces: Doxygen, docstrings, type signatures, generated references, all automated, all about how to call the thing
- We automated almost nothing about why it exists, why it works that way, or what was rejected
- Architecture decision records exist, and are rare
- Where the why survives at all, it survives by accident, in a commit message or a review thread nobody will find again

The formal documentation identifies what, and if you're lucky, how. It never covers why.

The history is the record

- `git log` on a file or directory, to see the shape of the change: what arrived together, and how often
- `git log -S`, the pickaxe, to find the commit where a string first appeared, which is usually the commit that explains it
- `git blame -w -C`, to get past reformatting and code movement to the change that actually wrote the line

Note:

- Squashed history, imported history and "fix build" destroy the record
- A commit says what changed, and only sometimes why

The constraint that made the change necessary is usually the thing nobody thought to write down

How AI can help

The Good

- A first pass over a subsystem: what is here, what calls what, what this function appears to do
- Explaining domain vocabulary, which is often the thing actually blocking you rather than the code
- Driving tools you have never used, the grep incantation, the Doxyfile, the pickaxe, and later the debugger
- It can suggest tools and approaches you didn't think of

Caveats

- The source shows every path that could run, with no indication of which ones do
- Common and rare look identical in text, so it has no sense of what is load bearing
- It will describe a branch that has never executed with the same confidence as the hot loop
- You need to verify any conclusions AI draws

Ultimately, it can tell you what the code might do. It cannot tell you what it did

MySQL 8.0.35: 4,343,511 lines of code

An unknown codebase, no one familiar with it to ask, and no idea why any of it is like this

```
chris@hyperion:~/tmp/mysql-8.0.35$ cloc .
35707 text files.
12819 unique files.
21827 files ignored.

github.com/AlDanial/cloc v 1.98 T=9.82 s (1305.6 files/s, 650515.5 lines/s)
-----
Language               files      blank      comment      code
-----
C++                     4467       437653       561364       2384569
C/C++ Header           5211       200309       511638       895020
C                       635        59973        75534       341537
Text                   186        10159         0         164174
JSON                    70          3           0         115239
Pascal                  260       17800       35508       103489
Bourne Shell            122       15026       17801       94354
Java                    534       12915       21441       55213
m4                      20        3950         962       36727
CMake                   451        6030       14147       36434
SQL                     203        2307        6319       26634
Perl                    122        7794        5270       25205
----- Snip -----
SUM:                   12819       781666     1261752     4343511
-----
```



x60

A bug nobody had explained

- Bug [#113097](#): Opened November 2023, reproduced within hours, then left on the tracker with no explanation
- One row in the table. Change almost anything and the query works

```
mysql> CREATE TABLE temp (createdAt datetime, timezone varchar(64));  
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> INSERT INTO temp VALUES ('2022-02-15 16:47:12', null);  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT * FROM (  
->   SELECT coalesce(timezone, '+00:00') tz,  
->           convert_tz(createdAt, '+00:00', (select tz)) ca,  
->           if(createdAt, if((select 1) = 1, 1, 1), 1) random  
->   FROM temp  
-> ) aux  
-> WHERE ca = '2022-02-15 16:47:12';  
Empty set (0.00 sec)
```

Reporter: "I have no idea what is going on.. I imagine some race conditions 🙄"

AI as an interface

- **Domain** What is this SQL meant to do? What is `convert_tz` for? What is derived-table condition pushdown?
- **Tooling** How do I build MySQL and reproduce this issue? How do I get MySQL to tell me what it did? Optimizer trace, and how to run the test suite against a patched build
 - The optimizer trace has been in the server for years, but I did not know it existed
- **Where to look** What state is worth capturing, and at what point in the execution?

It got me diagnostic output, and helped me try to figure it out

AI: Confident, fluent, and wrong

- I asked the obvious question: Why is this returning nothing?
- A root cause in `replace_columns_in_cond()`, and a patch for `Item_func_convert_tz::get_date()`. Wrong function, wrong mechanism, wrong fix
- Second attempt, I provided my own theory: `(select tz)` replaced by a bare `tz` during pushdown. Eighteen minutes later, a forty-line patch to `Item_field::is_valid_for_pushdown`
- The transcript shows it wondered three times whether `has_subquery()` was misbehaving. It never checked and wrote the patch anyway

Every answer was specific, plausible and wrong. How would I know?

Who knows what?

Why is it like this?

What does it actually do?

What only comes out at runtime

Run it

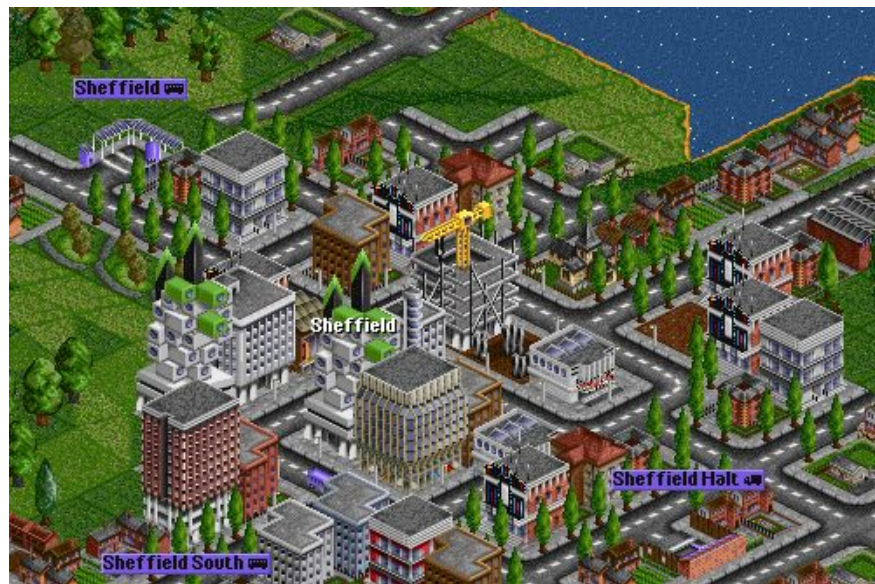
- The best way to start understanding what the code does is to watch it do it
- Reading shows you every path that could execute. Running shows you the one that did
- Obvious, and skipped constantly, because reading feels like easy progress and running requires setup
- Build systems are the usual blocker, but AI is genuinely good at this



Everything after this starts from a program that is actually running

What running tells you that reading cannot

- Which paths are real, and which ones never execute at all
- What the values actually are, rather than what the types allow
- Where the time goes, and what gets called ten thousand times
- Build with debug information or you get none of it: `cmake -DCMAKE_BUILD_TYPE=Debug`, and third party libraries too



A nice write up on Debug information is here:

- <https://developers.redhat.com/articles/2022/01/10/gdb-developers-gnu-debugger-tutorial-part-2-all-about-debuginfo>
- <https://undo.io/resources/guide-to-symbols-debug-info/>

Without debug information the program still runs, it is just harder to look inside 55

A debugger

- Stop the program where something interesting happens, and look at what it actually holds
- **Breakpoints** stop on a line. **Watchpoints** stop when a value changes
- **b, c, n, s, f, bt**. Six commands will get you a long way
- You can attach to something already running, which matters when restarting it is not an option
- AI can be good at driving these, but struggles when things get complicated and stateful

GDB cheatsheet - page 1	
Running	<where>
# gdb <program> (core dump) Start GDB (with optional core dump).	function_name Break/watch the named function.
# gdb --args <program> <args...> Start GDB and pass arguments	line_number Break/watch the line number in the current source file.
# gdb --pid <pid> Start GDB and attach to process.	file:line_number Break/watch the line number in the named source file.
set args <args...> Set arguments to pass to program to be debugged.	Conditions
run Run the program to be debugged.	break/watch <where> if <condition> Break/watch at the given location if the condition is met. Conditions may be almost any C expression that evaluate to true or false.
kill Kill the running program.	condition <breakpoint#> <condition> Set/change the condition of an existing break- or watchpoint.
Breakpoints	Examining the stack
break <where> Set a new breakpoint.	backtrace where Show call stack.
delete <breakpoint#> Remove a breakpoint.	backtrace full where full Show call stack, also print the local variables in each frame.
clear Delete all breakpoints.	frame <frame#> Select the stack frame to operate on.
enable <breakpoint#> Enable a disabled breakpoint.	Stepping
disable <breakpoint#> Disable a breakpoint.	step Go to next instruction (source line), diving into function.
Watchpoints	
watch <where> Set a new watchpoint.	
delete/enable/disable <watchpoint#> Like breakpoints.	

next Go to next instruction (source line) but don't dive into functions.	Variables and memory
finish Continue until the current function returns.	print/format <what> Print content of variable/memory location/register.
continue Continue normal execution.	display/format <what> Like „print“, but print the information after each stepping instruction.
	undisplay <display#> Remove the „display“ with the given number.
	enable display <display#> disable display <display#> En- or disable the „display“ with the given number.
	x/nfu <address> Print memory. n: How many units to print (default 1). f: Format character (like „print“). u: Unit. Unit is one of: b: Byte, h: Half-word (two bytes) w: Word (four bytes) g: Giant word (eight bytes)).

© 2007 Marc Haisenko <marc@darkdust.net>

GDB Cheat sheet (page 1 of 2)

<https://darkdust.net/files/GDB%20Cheat%20Sheet.pdf>

A debugger gives you the state and can follow control flow


```
chris@hypon:~/Undo/DeM x chris@hypon:~/Undo/DeM x chris@hypon:~/Undo/DeM x
recording 1,036,907,458> p *(const Vehicle*)$train
$20 = {<PoolVehicle, unsigned int, 512, 1044480, (PoolType)1, false, true>:PoolItem<&_vehicle_pool> = {index = 7}, <BaseVehicle> = {type = VEH_BEGIN}, <BaseConsist> = {
  _vptr.BaseConsist = 0x55bc702ea840 <vtable for Train16>, name = "", current_order_time = 712, lateness_counter = 0, timetable_start = 0, depot_unbunching_last_departure = 0,
  depot_unbunching_next_departure = 0, round_trip_time = 0, service_interval = 150, cur_real_order_index = 1 '\001', cur_implicit_order_index = 1 '\001', vehicle_flags = 9},
  next = 0x55bcac07bfc0, previous = 0x0, first = 0x55bcac4900c0, next_shared = 0x0, previous_shared = 0x0,
  tile = {<StrongTypedefBase> = {<No data fields>}, <StrongType::Compare::mixin<StrongType::Typedef<unsigned int, TileIndexTag, StrongType::Compare, StrongType::Integer, StrongType::Compatible<int>,
  StrongType::Compatible<long>>, unsigned int>> = {<No data fields>}, <StrongType::Integer::mixin<StrongType::Typedef<unsigned int, TileIndexTag, StrongType::Compare, StrongType::Integer, StrongType
::Compatible<int>, StrongType::Compatible<long>>, unsigned int>> = {<No data fields>}, <StrongType::Compatible<int>::mixin<StrongType::Typedef<unsigned int, TileIndexTag, StrongType::Compare, Stron
gType::Integer, StrongType::Compatible<int>, StrongType::Compatible<long>>, unsigned int>> = {<No data fields>}, <StrongType::Compatible<long>::mixin<StrongType::Typedef<unsigned int, TileIndexTag,
  StrongType::Compare, StrongType::Integer, StrongType::Compatible<int>, StrongType::Compatible<long>>, unsigned int>> = {<No data fields>}, value = 28476},
  dest_tile = {<StrongTypedefBase> = {<No data fields>}, <StrongType::Compare::mixin<StrongType::Typedef<unsigned int, TileIndexTag, StrongType::Compare, StrongType::Integer, StrongType::Compatible<
  int>, StrongType::Compatible<long>>, unsigned int>> = {<No data fields>}, <StrongType::Integer::mixin<StrongType::Typedef<unsigned int, TileIndexTag, StrongType::Compare, StrongType::Integer, Stron
gType::Compatible<int>, StrongType::Compatible<long>>, unsigned int>> = {<No data fields>}, <StrongType::Compatible<int>::mixin<StrongType::Typedef<unsigned int, TileIndexTag, StrongType::Compare,
  StrongType::Integer, StrongType::Compatible<int>, StrongType::Compatible<long>>, unsigned int>> = {<No data fields>}, <StrongType::Compatible<long>::mixin<StrongType::Typedef<unsigned int, TileInde
xTag, StrongType::Compare, StrongType::Integer, StrongType::Compatible<int>, StrongType::Compatible<long>>, unsigned int>> = {<No data fields>}, value = 30012}, profit_this_year = {
  m_value = 4391085}, profit_last_year = {m_value = 0}, value = {m_value = 19583}, cargo_payment = 0x0, coord = {left = 6456, top = 10716, right = 6543, bottom = 10787},
  hash_viewport_next = 0x7f43701fe370, hash_viewport_prev = 0x7f43701f9d80, hash_tile_next = 0x0, hash_tile_prev = 0x55bc705568e0 <_vehicle_tile_hash+114144>,
  hash_tile_current = 0x55bc705568e0 <_vehicle_tile_hash+114144>, colourmap = 781,
  build_year = {<StrongTypedefBase> = {<No data fields>}, <StrongType::Compare::mixin<StrongType::Typedef<int, TimerGame<Calendar>::YearTag<Calendar>, StrongType::Compare, StrongType::Integer>, int>
  > = {<No data fields>}, <StrongType::Integer::mixin<StrongType::Typedef<int, TimerGame<Calendar>::YearTag<Calendar>, StrongType::Compare, StrongType::Integer>, int>> = {<No data fields>},
  value = 1950},
  age = {<StrongTypedefBase> = {<No data fields>}, <StrongType::Compare::mixin<StrongType::Typedef<int, TimerGame<Calendar>::DateTag<Calendar>, StrongType::Compare, StrongType::Integer>, int>> = {<N
  o data fields>}, <StrongType::Integer::mixin<StrongType::Typedef<int, TimerGame<Calendar>::DateTag<Calendar>, StrongType::Compare, StrongType::Integer>, int>> = {<No data fields>}, value = 267},
  economy_age = {<StrongTypedefBase> = {<No data fields>}, <StrongType::Compare::mixin<StrongType::Typedef<int, TimerGame<Economy>::DateTag<Economy>, StrongType::Compare, StrongType::Integer>, int>>
  = {<No data fields>}, <StrongType::Integer::mixin<StrongType::Typedef<int, TimerGame<Economy>::DateTag<Economy>, StrongType::Compare, StrongType::Integer>, int>> = {<No data fields>},
  value = 267},
  max_age = {<StrongTypedefBase> = {<No data fields>}, <StrongType::Compare::mixin<StrongType::Typedef<int, TimerGame<Calendar>::DateTag<Calendar>, StrongType::Compare, StrongType::Integer>, int>> =
  {<No data fields>}, <StrongType::Integer::mixin<StrongType::Typedef<int, TimerGame<Calendar>::DateTag<Calendar>, StrongType::Compare, StrongType::Integer>, int>> = {<No data fields>},
  value = 7320},
  date_of_last_service = {<StrongTypedefBase> = {<No data fields>}, <StrongType::Compare::mixin<StrongType::Typedef<int, TimerGame<Economy>::DateTag<Economy>, StrongType::Compare, StrongType::Intege
  r>, int>> = {<No data fields>}, <StrongType::Integer::mixin<StrongType::Typedef<int, TimerGame<Economy>::DateTag<Economy>, StrongType::Compare, StrongType::Integer>, int>> = {<No data fields>},
  value = 712465},
  date_of_last_service_newgrf = {<StrongTypedefBase> = {<No data fields>}, <StrongType::Compare::mixin<StrongType::Typedef<int, TimerGame<Calendar>::DateTag<Calendar>, StrongType::Compare, StrongType
  e::Integer>, int>> = {<No data fields>}, <StrongType::Integer::mixin<StrongType::Typedef<int, TimerGame<Calendar>::DateTag<Calendar>, StrongType::Compare, StrongType::Integer>, int>> = {<No data fie
  lds>}, value = 712465}, reliability = 56036, reliability_spd_dec = 80, breakdown_ctr = 0 '\000', breakdown_delay = 0 '\000', breakdowns_since_last_service = 0 '\000',
  breakdown_chance = 208 '\320', x_pos = 968, y_pos = 1780, z_pos = 56, direction = DIR_SE, owner = OWNER_BEGIN, spritenum = 1 '\001', x_extent = 3 '\003', y_extent = 7 '\a', z_extent = 6 '\006',
  x_bb_offs = 0 '\000', y_bb_offs = -1 '\377', x_offs = -1 '\377', y_offs = -3 '\375', engine_type = 9, fill_percent_te_id = 65535, unitnumber = 1, cur_speed = 126, subspeed = 40 '(',
  acceleration = 12 '\f', motion_counter = 1523150, progress = 80 'P', waiting_triggers = 6 '\006', random_bits = 48247, last_station_visited = 0, last_loading_station = 0,
  last_loading_tick = 25620, cargo = {<CargoListVehicleCargoList, std::cxx11::list<CargoPacket*, std::allocator<CargoPacket*>> >> = {count = 0, cargo_periods_in_transit = 0,
  packets = empty std::cxx11::listPython Exception <class 'AttributeError': 'NoneType' object has no attribute 'pointer'
  }, feeder_share = {m_value = 0}, action_counts = {0, 0, 0, 0}, cargo_type = 0 '\000', cargo_subtype = 0 '\000', cargo_cap = 0, refit_cap = 0,
  cargo_age_counter = 16, trip_occupancy = 42 '*', day_counter = 11 '\v', tick_counter = 68 'D', running_ticks = 34 '!', load_unload_ticks = 20, vehstatus = 8 '\b', subtype = 9 '\t',
  current_order = {<PoolOrder, unsigned int, 256, 16711680, (PoolType)1, false, true>:PoolItem<&_order_pool> = {index = 1}, type = 81 'Q', flags = 0 '\000', dest = 1, refit_cargo = 254 '\376',
  wait_time = 148, travel_time = 962, max_speed = 65535, next = 0x0}, {orders = 0x55bcac07b610, old_orders = 0x55bcac07b610}, grf_cache = {position_consist_length = 0,
  position_same_id_length = 0, consist_cargo_information = 0, company_information = 0, position_in_vehicle = 0, cache_valid = 0 '\000'}, vcache = {cached_max_speed = 128,
  cached_cargo_age_period = 185, cached_vis_effect = 20 '\024', group_id = 65534, sprite_cache = {last_direction = DIR_SE, revalidate_before_draw = false, is_viewport_candidate = true,
  old_coord = {left = 6448, top = 10712, right = 6535, bottom = 10783}, sprite_seq = {seq = {sprite = 2916, pal = 0}, {sprite = 1878187796, pal = 21948}, {sprite = 2890465472, pal = 21948}, {
  sprite = 26112, pal = 26172}, {sprite = 1, pal = 26428}, {sprite = 65538, pal = 0}, {sprite = 0, pal = 0}, {sprite = 3, pal = 0}}, count = 1}}
recording 1,036,907,458> watch -l (
```

STEP BY STEP DEBUGGING



MONKEYUSER.COM

Start from the failure, not from a guess

- A breakpoint is a guess about where the problem is, made before you know what the problem is
- By the time you can see that something is wrong, whatever caused it has already happened
- Recording the run gives you a file: the same execution, opened as often as you like, walked in either direction
- Reverse debugging has been in gdb since 7.0, so this is a category of tool rather than one product



You stop asking the program to fail again,
and start asking the recording what it did

Back to the empty set

- One row. A query that plainly matches it. Nothing comes back
- Reproduced by Oracle, then silence on the tracker
- Three explanations from an AI, each specific, each plausible, each wrong

```
mysql> CREATE TABLE temp (createdAt datetime, timezone varchar(64));  
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> INSERT INTO temp VALUES ('2022-02-15 16:47:12', null);  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT * FROM (  
->   SELECT coalesce(timezone, '+00:00') tz,  
->           convert_tz(createdAt, '+00:00', (select tz)) ca,  
->           if(createdAt, if((select 1) = 1, 1, 1), 1) random  
->   FROM temp  
-> ) aux  
-> WHERE ca = '2022-02-15 16:47:12';  
Empty set (0.00 sec)
```

So how do I find out what it really did?

Where do you start when nothing happened?

- The row was never produced, so there is no moment of failure to break on
- I did not know how MySQL executes a query, or where a query even begins
- What I did know is that an empty answer left the process, so I started at the boundary and worked inwards
- Break where the reply is sent, then read the call stack. That backtrace is the architecture document nobody wrote
- AI as a sounding board throughout: what is this function for, what is this structure, is that a reasonable next place to look

The last thing a program does is always a real event, and you can always stand on it

What it turned out to be

- `convert_tz` ran once, and returned `NULL`, so the comparison rejected the only row
- The condition had been pushed down into the derived table, and the expression was cloned to move it
- The clone was made by printing the expression back to SQL text and re-parsing it. The printed text just said `tz`, not the `coalesce` it represented, so the meaning was already gone as `tz` was not available to be dereferenced at that point
- I wrote in the tracker that the bare `tz` was "erroneously resolved" to `temp.timezone`. I knew it happened, but I still had no idea how.

<https://bugs.mysql.com/bug.php?id=113097>

The source said `coalesce`. The execution said `timezone`. Only one of those was checkable

2026: The same AI, made to answer to the recording

- Same class of model that was confidently wrong an hour into this bug
- The difference is not a better model. It is that it cannot assert anything it has not checked
- It drives the recording itself: navigate to a point in time, log calls to a function, inspect state, trace a value back to the instruction that wrote it
- Every claim carries a coordinate in the recording, so anyone can go and look
- A second agent is told to verify the diagnosis: find the gaps, the assumptions, the claims that are not evidenced. It has to approve before the answer counts

Context and Constraint to hone Intelligence

The prompt, typos and all

"in build/recordings is a undo recording of a bug being see in this version of mysql. Use this recording and the codebase to diagnose this error. The Notes.txt in the root of the workspace gives insight in to how the bug is reproduced, and the build/record-issue.sh script shows how the recording was generated. This is a test of your ability to root cause this issue from the recording, description, codebase, and the error alone. Be sure to use any and all Undo tools available from the MCP server to tackle this recording. Also, do not look outside this workspace for hints, this is intended to test your capability as if this was a live bug in a uptodate code base, so everything you should need is in this folder"

The review came back APPROVED – the reviewer independently verified the source code claims (Item_ref::print, copy_field_info, parse_expression/clone_expression) and confirmed the causal chain is complete. Here is the final diagnosis.

Root cause

The empty result is caused by a bug in derived-table condition pushdown: when the outer WHERE ca = '2022-02-15 16:47:12' is pushed down into the derived table, the cloning machinery silently rewrites convert_tz's third argument from (select tz) – meaning coalesce(timezone, '+00:00') – into the raw nullable base column temp.timezone. Since timezone is NULL in the row, convert_tz returns NULL, the pushed-down filter rejects the only row, and the query returns an empty set.

The causal chain (all verified in the recording, thread 39)

1. Setup during resolution. The scalar subquery (select tz) was transformed by Item_singlerow_subselect::select_transformer into a plain Item_ref (named "(select tz)") pointing at the select-list item coalesce(...) AS tz. Verified: the ref's m_ref_item resolves to an Item_func_coalesce named tz.

2. The corrupting clone (bbcount 51,452,192 → 51,456,047). During Query_block::push_conditions_to_derived_tables, Item_field::replace_with_derived_expr (sql/item.cc:1069) clones the derived expression for ca by printing it to text and re-parsing it (Query_block::clone_expression → parse_expression, sql/sql_derived.cc:755/640). Two flaws combine:

- Item_ref::print (sql/item.cc:8373-8377) prints an alias-of-expression reference as just the bare alias identifier. The captured printed text in the recording is literally: convert_tz(`testing`.`temp`.`createdAt`, '+00:00', `tz`)

The coalesce/subquery semantics are already gone from the text; `tz` re-parses as a bare column Item_field (bookmark clone-reparse-text-lost-coalesce).

- copy_field_info (sql/sql_derived.cc:498-559) then walks the original tree collecting Item_field metadata in prefix order – and this walk descends through the ref into coalesce(timezone, '+00:00'), collecting [createdAt, timezone] – and grafts that metadata positionally onto the clone's fields [createdAt, tz]. At sql_derived.cc:553 the clone's tz item gets field = temp.timezone (bookmark copy_field_info-grafts-timezone-onto-tz; traced the exact write, prior value 0x0). The trees happen to have two fields each, so the debug assert never fires, and resolve_expression's "short cut fix_fields" trusts the pre-set field info, so the name/field mismatch (item_name = "tz", field = timezone) is never detected.

3. The failure at execution (bbcount 51,815,865, bookmark convert_tz-get_date-to_tz-null). Inside MaterializeIterator → FilterIterator::Read, Item_func_convert_tz::get_date (sql/item_timefunc.cc:2410) runs the pushed-down condition. args[2]→val_str_ascii() reads temp.timezone = NULL, so my_tz_find(thd, NULL) returns nullptr, to_tz == nullptr, and get_date returns NULL. NULL = '2022-02-15 16:47:12' is not true → the row is filtered out before it ever reaches the materialized derived table → Empty set.

Why the design is broken

clone_expression's print-and-reparse approach assumes the reparsed tree's column references correspond 1:1, in traversal order, with the original tree's Item_fields. Any Item_ref that is an alias of another select-list expression breaks that invariant: it prints as one bare identifier but walks as the fields inside the aliased expression. copy_field_info then silently rebinds the alias to whatever base column happens to sit at the same position – here degrading coalesce(timezone, '+00:00') to timezone.

Fix direction

Either refuse to push a condition down when the replacement expression contains a reference to another select-list alias (the pushdown-eligibility check should treat such refs as unsafe), or make the clone faithful – print the ref as its full underlying expression (and make copy_field_info robust to ref targets) so the pushed condition evaluates coalesce(timezone, '+00:00') as written. The conservative eligibility-check fix is the safer patch for 8.0.

Bookmarks left in the recording as evidence: clone-reparse-text-lost-coalesce (51,452,192), copy_field_info-grafts-timezone-onto-tz (51,456,047), convert_tz-get_date-to_tz-null (51,816,424).

My fix, Oracle's fix, and the agent's

```
if (m_ref_item == nullptr)           // unresolved reference: print reference
    return Item_ident::print(thd, str, query_type);

- if (!const_item() && m_alias_of_expr &&
*   (ref_item()->type() != Item::FUNC_ITEM || !thd->lex->reparse_derived_table_condition) &&
+ if (!thd->lex->reparse_derived_table_condition && m_alias_of_expr &&
    ref_item()->type() != Item::CACHE_ITEM && ref_type() != VIEW_REF &&
    table_name == nullptr && item_name.ptr()) {
    Simple_cstring str1 = ref_item()->real_item()->item_name;
    append_identifier(thd, str, str1.ptr(), str1.length()); // prints `tz`
}
```

- My proposed fix: Keep Red, add Green - Specific to case seen in the bug
- MySQL 8.0.36 fix: Remove Red, add Purple - More general fix
- Undo AI path: none of the above. Refuse the pushdown, not the shortcut
 - Different location in `sql/item.cc` in `is_valid_for_pushdown`. Treat an alias-of-expression ref as unsafe to push, so the faulty clone is never made
 - It named faithful printing as the alternative and preferred the conservative option

How this generalises

- Agents are bad at claims nobody can check, and this was never a question of a better model
- Give one a source of truth it can interrogate, and it becomes an investigator rather than a storyteller
- Execution history is one such source. It is not the only one, and the principle matters more than the instrument
- On a DuckDB bug: static context alone took over an hour, cost about thirty-seven dollars and produced a confidently wrong root cause. With runtime context, twelve to twenty-four minutes, a few dollars, and the right answer
- Mark Williamson's poster makes the same argument systematically, benchmarked against closed-source bugs so public training data cannot skew the result

Confidently wrong is also slower and more expensive

Mark Williamson's poster from CppCon 2026

AUTONOMOUS ROOT CAUSE ANALYSIS WITH FRONTIER MODELS

WHAT IS RUNTIME CONTEXT? AND WHY IS IT MORE IMPORTANT THAN USING THE LATEST MODEL?

Mark Williamson, Nick Read, Finn Grimwood, Ed Tai, Marco Barisione - undo.

undo

Background

Tokenmaxxing is gone, long live Valuemaxxing
A year ago, AI mandates rolled out across major tech companies: just use AI, more is better.

Today, commercial software development routinely uses **coding agents** - with many developers now reporting that most of their code is written through models, not directly.

AI usage has since exploded, driving up spend on top models. **Focus is shifting from adoption to ROI**, with greater scrutiny on where AI spend goes and the value it delivers. Nobody wants to lose the edge AI tooling provides, but costs need to remain sustainable.

This is a balancing act! Engineering organisations must simultaneously:



Successful AI strategies need to balance these two, conflicting goals. Coarse organisational mandates like "prefer Sonnet to Opus" or "use no more than N tokens / mins" minimises investment costs but risk the productivity returns that motivated AI adoption.

Typing code isn't the bottleneck on development

Meanwhile, it is widely acknowledged that writing code is a small fraction of development work. Delivering (and maintaining) a working product requires a huge effort in **understanding software behaviour**, in particular **root cause analysis (RCA)** of unexpected behaviour.

This is equally true for new code as for existing code that's been there for decades. **Without sufficient AI gains in understanding runtime behaviour, Root Cause Analysis will become a progressively-worse bottleneck on the Software Development Lifecycle.**

The LLM isn't the bottleneck on intelligence

A coding agent's reasoning comes from the LLM (Large Language Model) it uses. Major new LLM releases are now announced monthly or even more frequently, so raw reasoning isn't in short supply. And yet, the LLM's power is only as good as the **context** it can read. Without the right inputs even the smartest model will fail to give a good answer.



Fig 1: 'To the best of the authors' knowledge there is (sadly) no Paris Marathon for Dogs - without access to web search this agent is confidently wrong

in other words

$$AI = MODEL + CONTEXT$$

Our work

Runtime context

In discussing AI's ability to reason about software behaviour, we find it helpful to divide the information it uses into two broad categories:



Static context is always available in a correctly-configured system. The quality of **runtime context** available depends on many factors: the detail in a bug report, the fidelity of built-in logging, etc. **Process recordings**, as by Undo, or Microsoft's TTD are an ideal form of this, containing a precise and complete representation of program state during a particular execution.

Runtime context is the bottleneck on RCA

We've already seen that providing the correct context is essential for LLM reasoning. To illustrate the power of runtime context in reasoning about software we'll consider what the most powerful frontier models can achieve when their context is limited.

We used our internal benchmark to test **Table 5** on DuckDB issue #20086, which involved unexpected output from an SQL query. DuckDB's codebase is just ~0.5M lines of code.

	Static Context (just code and bug report)	Runtime Context (re-runs, logging, process recording)
Elapsed time	> 1 hour	12 - 24 minutes
Cost	\$37.60	\$2.88 to \$6.31
Root cause	❓ (confidently wrong)	✅ (correct answer)

Note that the agent is always able to give a **confident** answer - they are very good at doing that! But when deprived of crucial information about the system's runtime behaviour, models are forced to speculate and provide a misleading results in return for the resources consumed.

Measuring the effectiveness of agentic RCA

We set out to build an objective measure of how well AI agents can perform **autonomous root cause analysis (RCA)** of non-trivial bugs in C and C++ codebases.

Fully autonomous RCA is the most challenging test available and can be measured objectively, without requiring a human in the loop providing feedback.

Non-trivial bugs are the ones that are most difficult to solve and least predictable to analyse. These are the ones that keep working software away from customers. These usually involve one (or more) of:



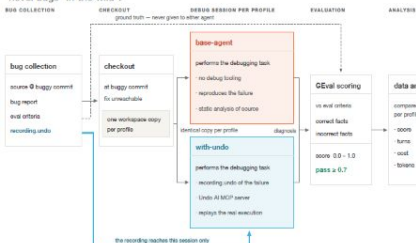
By their very nature, these bugs occur in **large codebases with complex invariants around complex state**. The root cause is typically far from the observed symptom, making it impossible to identify it from a simple backtrace or pre-defined logging.

Methodology

Bug library benchmark

Our goal is to test the ability of AI agents to reason about bugs involving complex state and control flow. Bugs are hand-selected to be specifically challenging for LLM reasoning and added to our "Bug Library" benchmark system.

Our goal is to simulate the process of resolving such a bug through reasoning about the code and its dynamic behaviour - the same challenge faced by engineers to investigate novel bugs "in the wild".



- Bug collection** - each bug is a real, historic defect, stored with its source, a symptoms-only report, ground-truth criteria and a recording of the failure
- Checkout** - the code is restored to just before the fix, with everything after made unreachable. This is to simulate a point in time of the codebase's history.
- Debug session per profile** - two agents get identical copies and the same task. The agents are given free reign; one with Undo AI and one without.
- Evaluation** - GEval used to evaluate presence of correct and incorrect information in returned summaries. More correct facts make for a higher score, with agents penalised for presence of known incorrect facts and red-herrings. Pass marks are >= 0.7.
- Analysis** - the two sides compared on score and on what it cost: turns, money, tokens

Do agents cheat at benchmarks?

AI agents are inventive, with a strong drive to provide an **answer to the question asked**. This makes them challenging to benchmark consistently, given their expertise in navigating computer systems and data.

These skills can easily undermine the **behaviour** we want to measure, which is the ability to reason about and explain a program's observed behaviour.

- git history used as a shortcut - for a recently (or artificially) introduced bug the changeset history can make a bug obvious. Agents will examine the current branch, search for other available branches and even access known remedies to gain knowledge.
- Bug information in the training corpus - for Open Source codebases, bug tracker data from pre-training may allow the model to answer using its innate knowledge instead of the tools provided. This effect appears more noticeable in larger models.
- Internet access - even without git, models will attempt to access reference copies of source code (obtaining download tools, if required) for injected bugs.

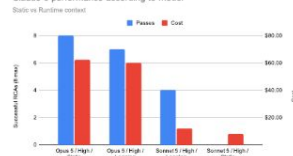
Findings

Model and runtime context

We benchmarked against closed-source bugs, to allow testing against recent models without their training from public data skewing our results. We used two current-generation models, Opus 5 (a model for coding, architecture and planning) and Sonnet 5 (a smaller, cheaper workhorse).

Here we compare the effect of pure **static context vs context from logging** and re-runs.

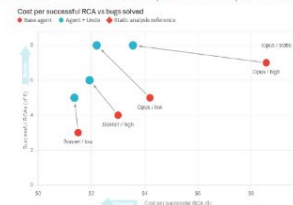
Claude 5 performance according to model



Opus is **significantly stronger at static analysis** than Sonnet and root causes issues in the (relatively small) codebases under test without needing logging / re-runs but the cost is high. For larger codebases we have seen its analysis is insufficient to solve bugs even with logging. Sonnet is **much cheaper but with limited effectiveness** with logs (and no success with static context).

Thinking effort and type of runtime context

Here, we compare the influence of the **high vs low effort** and the use of **logs vs Undo-based process recordings**. The models are allowed to run and re-run the software under test and to construct directed test cases as part of their analysis if needed.



We make several interesting observations from this data:

- Opus at High effort, with logs for runtime context is very effective at resolving issues. Adding Undo process recording retained this effectiveness, at lower cost.
- Opus at Low is cheaper than High for other runtime context source. It is significantly less effective with logging, whilst process recording retains effectiveness at even lower cost.
- Sonnet on High reasoning is cheaper but less effective than Opus on Low.
- Sonnet with access to Undo is cheaper and better than Opus with only logs
- Our market experience suggests process recording enables successful RCA in significantly larger codebases.

To truly maximise return on investment from AI tools it is not enough to look at model and token costs - you must maximise the quality of the context provided

Had it just read the answer?

- A directory listing in the first minute showed the folder above the workspace, with filenames revealing that a patch existed and that an earlier AI attempt had failed. I interrupted and added the boundary
- No file outside the workspace was ever read, those names appear once in the transcript, and the transcript shows how every step to `convert_tz` was derived from the recording
- The mechanism it found is unpublished. Not in my comment, not in the changelog, and Oracle's fix does not touch that function
- It recommended neither of the fixes you just saw, which is not what recitation looks like
- No transcript can prove the absence of latent knowledge. The clean version is a bug that postdates the cutoff, which is what the poster does

What else can you ask, and what answers it

The question	What answers it
Where does the time actually go	a sampling profiler, <code>perf record</code>
Who called whom, and how often	<code>callgrind</code> and <code>kcachegrind</code>
What did it ask the operating system for	<code>strace</code> , <code>ltrace</code>
What did it touch that it should not have	the sanitisers, <code>valgrind</code>
It is already running and I cannot restart it	attach to the live process
What did I learn last time I was here	your own notes

You do not need to know how to drive any of these. AI is generally reliable about the instrument, because a wrong invocation fails in seconds and you see it. It is not so reliable about the patient

None of this needs AI. It is all decades old and it all still works

See our talk last year at C++ on Sea for more information on some of these tools: <https://youtu.be/qmte-ZrzBKU>

Pro Tip: Keep a Journal

- Now that you have done a few investigations -- take some notes!
 - Making lists about what you learned or need to revisit will be useful!
- I recommend using whatever journaling tool is the lowest friction thing for you
 - e.g. physical notebook perhaps, but a 'Google Doc' or text file that is searchable by you, or AI, may be best
 - More on note taking systems (e.g. .plan files):
 - https://www.youtube.com/watch?v=S_W1fzrB-UEg

```
[idsoftware.com]
Login name: johnc In real life: John Carmack
Directory: /raid/nardo/johnc Shell: /bin/csh
Never logged in.
Plan:
```

This is my daily work ...

When I accomplish something, I write a * line that day.

Whenever a bug / missing feature is mentioned during the day and I don't fix it, I make a note of it. Some things get noted many times before they get fixed.

Occasionally I go back through the old notes and mark with a + the things I have since fixed.

--- John Carmack

```
= feb 18 =====
* page flip crap
* stretch console
* faster swimming speed
* damage direction protocol
* armor color flash
* gib death
* grenade tweaking
* brightened alias models
* nail gun lag
* dedicated server quit at game end
+ scoreboard
+ optional full size
+ view centering key
+ vid mode 15 crap
+ change ammo box on sbar
+ allow "restart" after a program error
+ respawn blood trail?
+ -1 ammo value on rockets
+ light up characters
```


Say what you verified, say what you assumed

- Every investigation has steps you checked and steps you skipped. In the write-up they look identical
- "Erroneously resolved" looks like an explanation, so nobody spotted the gap and asked how
- Mark the difference as you go. It costs one word
- The same discipline is what turned a confident hallucinator into a rigorous investigator, and it works on people first
- Leave a record of what you did and why, help us all get better

The gaps you name are the reason anyone believes the rest

Summary

- **Who knows what?** The people are the resource. Ask, write down what you are told, and make the next person's first week better than yours
- **Why is it like this?** Most code was the best answer available under constraints you cannot see. The documentation records what, and sometimes how. The why survives by accident
- **What does it actually do?** Run it, and go looking. Behaviour is the one account you can check

Intent is the fragile source of truth. Behaviour is the durable one

Douglas Adams - Dirk Gently's Holistic Detective Agency, 1987

"What I mean is that if you really want to understand something, the best way is to try and explain it to someone else.

That forces you to sort it out in your own mind.

And the more slow and dimwitted your pupil, the more you have to break things down into more and more simple ideas.

And that's really the essence of programming.

By the time you've sorted out a complicated idea into little steps that even a stupid machine can deal with, you've certainly learned something about it yourself.

The teacher usually learns more than the pupil."

From 1987. Still true today. I'm not sure the pupil is any less dimwitted.

**CONTEXT IS
IMPORTANT**

Thank you Cppcon 2026!



Code You Didn't Write

Understanding Large and Unfamiliar Codebases

Chris Croft-White
Staff Solutions Architect
chris@undo.io



Web: mshah.io
YouTube: www.youtube.com/c/MikeShah
Social: mikesah.bsky.social
Courses: courses.mshah.io
Talks: <http://tinyurl.com/mike-talks>

60 minutes | Intermediate Audience
13:30 - 14:30 Friday, Sept. 18, 2026

Supporting Resources

- Talks on Code Bases
 - Plenary: Coping With Other People's C++ Code, Laura Savino, CppCon 2023
 - <https://youtu.be/qyz6sOVON68>
- Debugging and Tools
 - Understanding Large and Unfamiliar Codebases, Mike Shah and Chris Croft-White, C++ on Sea 2025
 - <https://youtu.be/qmte-ZrzBKU>
 - Time Travel Debugging, Greg Law, Meeting C++ 2023
 - <https://youtu.be/qyGdk6QMpMY>
 - Back to Basics: Debugging in Cpp, Greg Law, CppCon 2023
 - <https://youtu.be/qgszy9GquRs>
 - Back to Basics: Debugging in C++, Mike Shah, CppCon 2022
 - <https://youtu.be/YzIBwqWC6EM>
 - Cool New Stuff in Gdb 9 and Gdb 10, Greg Law, CppCon 2021
 - <https://youtu.be/xSnetY3eoIk>
 - CppCon 2018: Greg Law "Debugging Linux C++"
 - <https://youtu.be/V1t6faOKjuQ>
 - CppCon 2016: Greg Law "GDB, A Lot More Than You Knew"
 - <https://youtu.be/-n9Fkq1e6sg>
 - Conditional Watchpoints to Follow The Data (OpenTTD video from earlier)
 - https://youtu.be/-CPoea_n7es
- Time Travel Case Studies
 - BeCPP Symposium 2026, Chris Croft-White, Agentic Time-Travel Debugging
 - https://youtu.be/unSghOeS1_o
 - Doom, reviving a zombie:
 - <https://youtu.be/tjJLZ1da6xs>
 - Quake 2: <https://www.jwhitham.org/2015/05/review-undodb-reversible-debugger.html>